

Question 1 — Technical Debt Diagnosis

Part 1: Category identification.

(a) Changing the rounding logic of “estimated delivery time” broke the unrelated “favorite restaurants” feature → **Entanglement (CACE)** — This shows that features in an ML system are not fully separate. A small change in one part can affect another part and change the model’s behavior.

(b) A marketing team silently reading the model’s raw output table → **Undeclared Consumers** — The marketing team was using the model output without officially declaring it as a dependency. This makes it difficult to safely change the output table because we may break something that depends on it.

(c) The training pipeline is 14 undocumented shell scripts with no orchestration → **Configuration & Glue-Code Debt** — There are many shell scripts, but there is no proper system managing them and no clear documentation. This creates a “pipeline jungle” where it is difficult to know which script should run, when it should run, and how everything is connected.

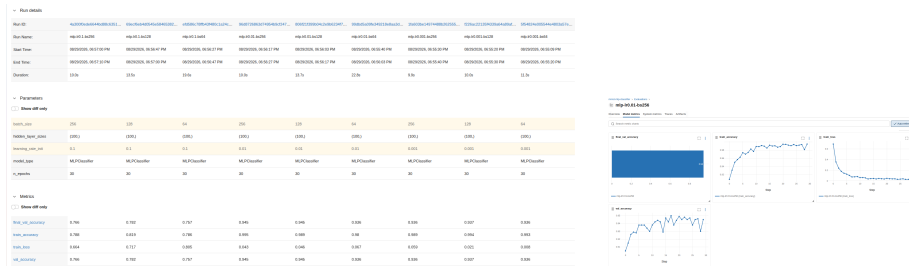
Mitigation (for c)). Replace the ad-hoc script chain with MLflow experiment tracking, using Git-pinned code. Each pipeline stage is run inside an MLflow run. All hyperparameters and configuration values are saved using `mlflow.log-param(...)`, and the run is linked to the exact Git commit using `mlflow.set-tag("git-commit", ...)`. This replaces the scattered and undocumented configuration with a single queryable, versioned source of truth. This means any engineer can easily find which configuration and code version were used to produce a particular model.

Question 2 — MLflow Experiment Comparison (MNIST + MLP)

Best-Performing Run. The best run was `mlp-lr0.01-bs256` (`learning_rate_init` = 0.01, `batch_size` = 256), which got a final validation accuracy of **0.945**, the same as `mlp-lr0.01-bs128`. Looking at the whole table below, both `lr=0.01` runs did better than `lr=0.001` (0.936–0.937) and much better than `lr=0.1` (0.757–0.782). This shows that `lr=0.01` was the right balance — fast enough to learn well, but not so fast that training became unstable.

Evidence of Overfitting. Looking at the graph for the best run below, the `train_loss` drops quickly in the first few epochs and becomes almost flat and close to zero by around epoch 8–10. The `train_accuracy` also rises fast and settles near 0.98. But `val_accuracy` rises quickly at first and then flattens out lower, around 0.93–0.95, and just goes up and down after that instead of improving. This gap between training and validation performance is a sign of overfitting — the model keeps getting better on training data but stops improving on new data. This same pattern is seen in all six `lr=0.01` and `lr=0.001` runs in the table, where `train_accuracy` (0.98–0.995) is always higher than `val_accuracy` (0.936–0.945) by 4 to 6 percent, and `train_loss` is very low (0.008–0.067) in every one of them.

Which Hyperparameter Has the Larger Effect. Comparing the two hyperparameters, `learning_rate_init` clearly has the bigger effect on performance. Changing it from 0.01 to 0.1 makes accuracy drop from about 0.94 to 0.76–0.78, no matter what `batch_size` is used. Changing `batch_size` alone only changes accuracy by about 1 percent. So learning rate matters much more than batch size in this experiment.



Question 3 — DVC Data Versioning & Rollback

DVC was initialized with an SSH remote; `filenames.csv` was pushed as v1 (1801 rows) and updated to v2 (2801 rows) after adding `new-labels.zip`. Rolling back with `git checkout v1 -- filenames.csv.dvc` + `dvc checkout` restored the file to exactly 1801 rows, an exact match to v1, as shown below. Full logs in `q3-dvc-versioning/`.

```
git checkout v1 -- filenames.csv.dvc
dvc checkout
dvc status
dvc diff
```

Question 4 — End-to-End Reproducibility Drill (as Partner B)

I reproduced my partner’s MLP-on-MNIST run using only `git clone`, `git checkout <commit>`, `conda env create -f environment.yml`, `dvc checkout`, and rerunning `train.py` with the same hyperparameters and seed — no other communication about environment or data. My run reproduced an accuracy of **0.9784** against the original **0.9782**, a difference of just **0.0002**, well within the stated tolerance of 0.005. I logged this outcome directly as a tag on the MLflow run, shown below, and committed the verification to the shared repository. Full logs, commit hashes, and screenshots are in `q4-reproducibility/` (shared repo linked in the README).

```
mlflow.log-param("reproducibility", "0.9784")
git commit -m "Reproduced Partner B's run with 0.9784 accuracy"
git push
```